

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 2

Total Marks:40

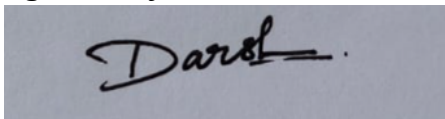
Annexure A

Industry Problem Solving Task

Code of Conduct:

*I Darshinie Karani.VM (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature appears to be 'Darsh' followed by a horizontal line and a dot.

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify

communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANSWERS

1.

Smart Hotel and Hospitality Management System

Role of Object-Oriented Principles

- Encapsulation: Reservation, Room, and Payment objects hide their internal state (booking status, room availability) behind controlled methods, preventing double-booking or inconsistent payment records.
- Inheritance: Standard Room, Deluxe Room, and Suite extend a common Room base class, allowing new room types to be added without rewriting reservation logic.
- Polymorphism: Room and service objects are processed through a shared interface (e.g., `calculatePrice()`, `reserve()`), so room-type-specific or service-specific behavior is handled without changing caller code.
- Abstraction: The service layer exposes operations like `bookRoom()` and `processCheckOut()` while hiding the pricing and availability logic behind them.

Contribution of the Layered Architecture

- Presentation Layer: Handles guest-facing booking screens and staff dashboards, forwarding requests to the service layer without direct database access.
- Service Layer coordinates reservation processing, check-in/check-out, and payment workflows, applying pricing and availability rules.
- Domain Layer models Guest, Reservation, Room, and Payment entities and their business logic.
- Data Access Layer connects to the OODBMS to persist and retrieve hotel objects.

The service layer acts as the coordination point for reservation processing: it validates room availability (domain layer), applies the correct pricing strategy, creates the reservation, and triggers persistence through the Repository — all without the presentation layer needing to know these internal steps. This keeps the booking workflow reusable across channels (front desk, mobile app, travel-agent portal) and easy to maintain as new services are added.

Contribution of Design Patterns

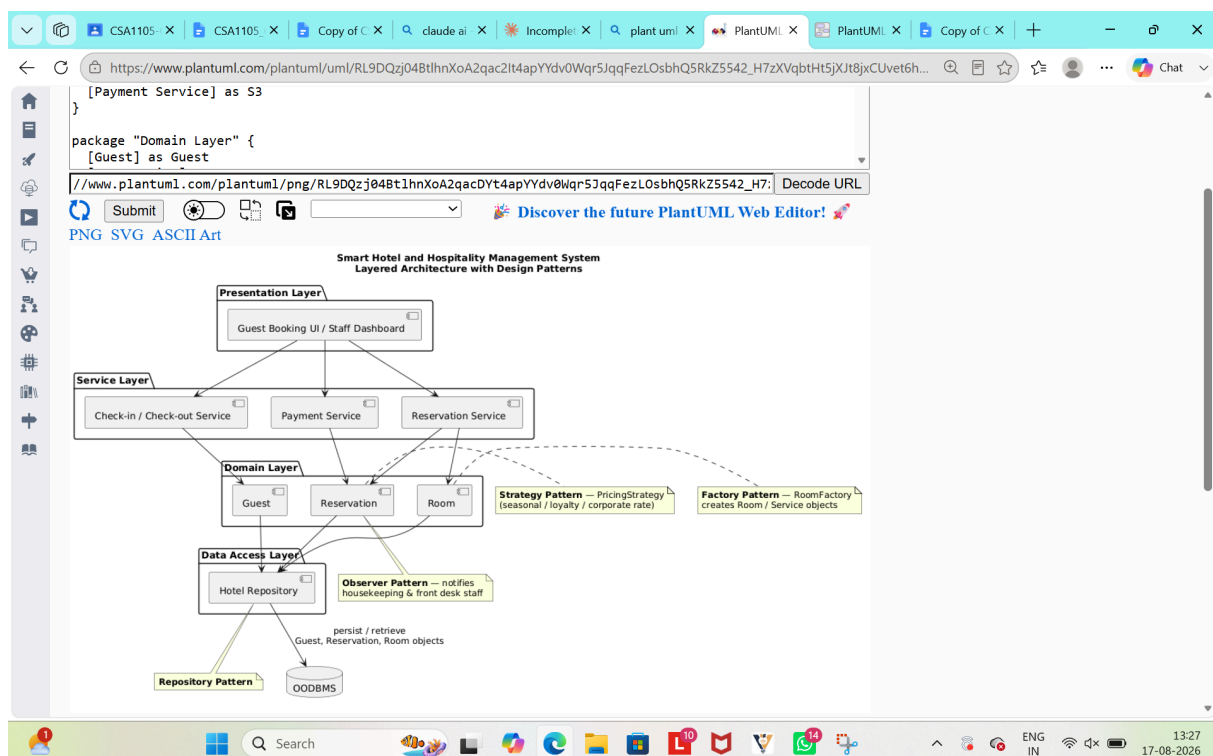
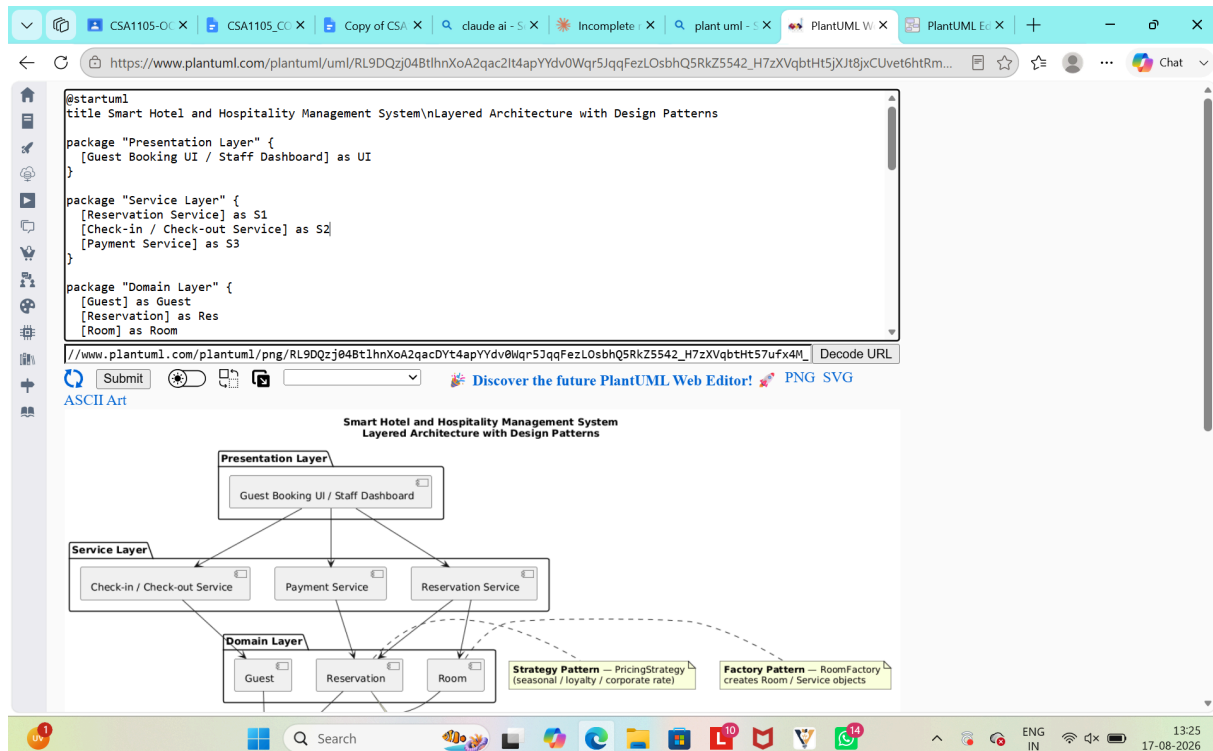
- Strategy Pattern: Encapsulates different pricing and discount policies (seasonal rate, loyalty discount, corporate rate) as interchangeable strategies, so new pricing rules can be added without modifying the reservation logic.
- Factory Pattern: Centralizes creation of room and service objects (spa service, room service), letting new room/service types be introduced by extending the factory rather than the reservation code.
- Observer Pattern: Notifies relevant staff or systems (housekeeping, front desk) when a reservation is made or a service is requested, decoupling notification logic from the core reservation process.
- Repository Pattern: Simplifies communication between the service layer and persistent hotel objects, hiding OODBMS query details behind a simple interface (e.g., `findAvailableRooms()`).

Communication Between Layers and the OODBMS

- When a reservation is confirmed in the service layer, the Repository is called to persist the Reservation, Guest, and updated Room-availability objects to the OODBMS.

- Because the OODBMS stores objects natively, a Reservation object (with its associated Guest, Room, and Payment references) can be retrieved as a complete, ready-to-use object graph rather than reassembled from multiple relational tables.
- The Repository exposes object-based retrieval methods (e.g., `getReservationsByDate()`) that the service layer uses for check-in/check-out and reporting, keeping OODBMS access confined to one layer.
- This isolation means future enhancements like smart-room automation can be added at the domain/service level without altering how reservation data is stored or retrieved.

Using Plant UML tool , we constructed the diagram



Conclusion

OOP principles make room, guest, and reservation objects reusable and self-contained, the layered architecture gives reservation processing a clear coordination point in the service layer, and the Strategy, Factory, Observer, and Repository patterns keep pricing, room/service creation, notifications, and persistence flexible and loosely coupled. The Repository's object-based interaction with the OODBMS ensures reservation data remains consistent and easy to retrieve as the hotel chain scales and adds smart-room features.

2.

Smart Logistics and Delivery Management System

Role of Object-Oriented Principles

- Encapsulation: Shipment, Vehicle, and DeliveryStatus objects hide their internal state (location, load capacity) behind controlled methods, ensuring status updates happen consistently.
- Inheritance: Standard Shipment, Express Shipment, and Fragile Shipment extend a common Shipment base class, so new shipment types can be added without altering existing tracking logic.
- Polymorphism: Shipment and route objects share a common interface (e.g., calculateDeliveryTime(), assignVehicle()), letting shipment-specific or route-specific behavior vary without changing calling code.
- Abstraction: The business layer exposes operations like scheduleDelivery() while hiding route optimization and vehicle-allocation detail.

Contribution of the Layered Architecture

- Presentation Layer: Provides dashboards for shipment registration and delivery tracking, forwarding all requests to the business layer.
- Business Layer coordinates shipment registration, vehicle allocation, and delivery-status updates, applying routing and scheduling rules.
- Domain Layer models Shipment, Vehicle, Warehouse, and Delivery entities and their business logic.
- Data Access Layer connects to the OODBMS to persist and retrieve shipment and vehicle objects.

Because vehicle allocation, route selection, and tracking logic live in the business/domain layers, the presentation layer (dashboards, customer notification screens) can change independently of how deliveries are actually planned and tracked — supporting adaptability. New warehouse or delivery-vehicle types can be introduced as new domain classes without disturbing existing tracking and notification code, supporting scalability and maintainability as the network grows.

Contribution of Design Patterns

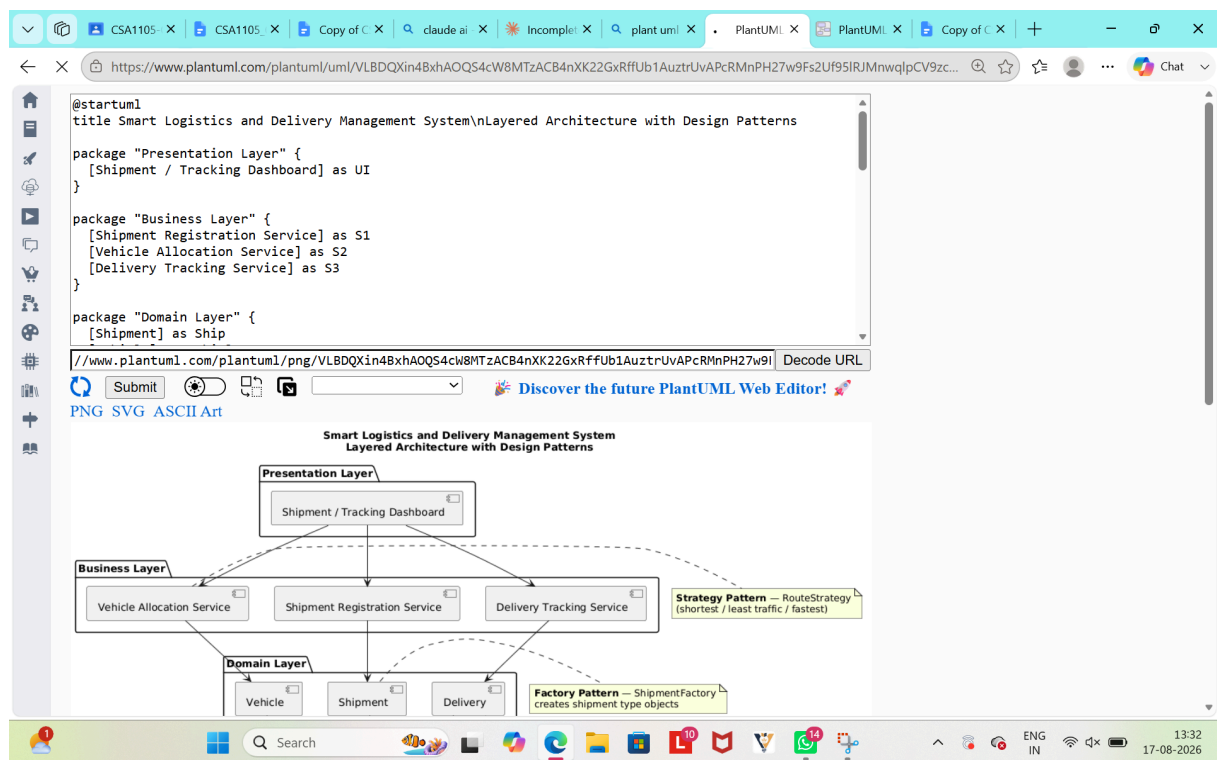
- Strategy Pattern: Encapsulates different route-selection algorithms (shortest distance, least traffic, fastest delivery) as interchangeable strategies, so new routing logic — including future AI-based optimization — can be added without changing the delivery-scheduling code.
- Factory Pattern: Centralizes creation of different shipment type objects, letting new shipment categories be added by extending the factory rather than modifying existing shipment-handling logic.

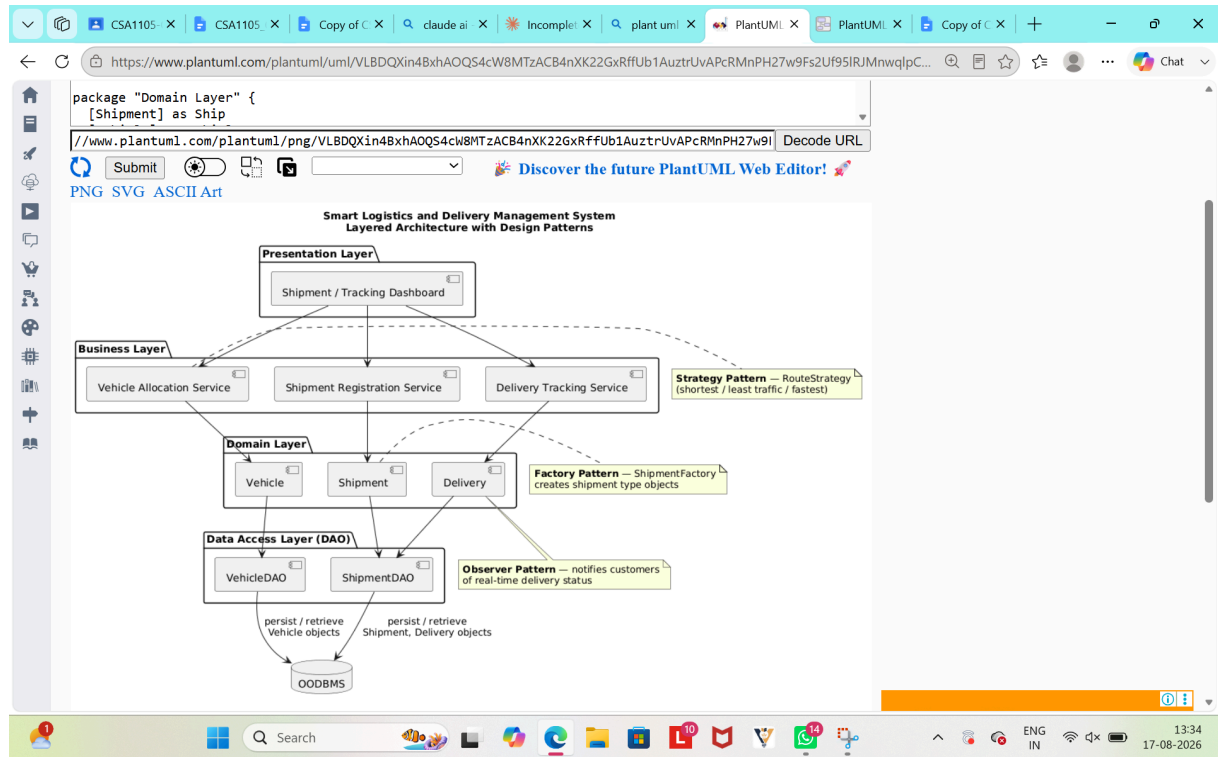
- Observer Pattern: Notifies customers and warehouse staff in real time when a delivery status changes, decoupling notification logic from the core shipment-tracking process.
- DAO Pattern: Manages persistent shipment and vehicle information through dedicated data-access objects, isolating OODBMS operations from business logic.

Communication Between Layers and the OODBMS

- When a shipment's status changes in the business layer, the domain layer updates the Shipment object and passes it to the DAO layer, which persists the change to the OODBMS.
- The OODBMS stores Shipment and Vehicle objects directly, so complex relationships (a shipment's assigned vehicle, route, and current warehouse) are retrieved as a connected object graph rather than reconstructed through relational joins.
- DAO objects expose methods like `getActiveShipments()` or `getAvailableVehicles()`, which the business layer uses for allocation decisions and tracking updates.
- Because only the DAO layer touches the OODBMS, future integration with autonomous delivery vehicles or AI-based route optimization can be added at the business/domain level without changing how shipment data is stored.

Using Plant UML , we constructed the diagram





Conclusion

OOP principles keep shipment and vehicle objects consistent and reusable, the layered architecture separates tracking dashboards from routing and allocation logic, and the Strategy, Factory, Observer, and DAO patterns give the system adaptable extension points for new routing algorithms, shipment types, and real-time notifications. The DAO layer's object-based interaction with the OODBMS keeps shipment, vehicle, and delivery data consistent and ready for future autonomous and AI-driven capabilities.